

OSProject: Monitor Web de Recursos, Aplicaciones y Procesos para GNU/Linux

Steven David Egas Jacome
Universidad de las Fuerzas Armadas ESPE
sdgegas@espe.edu.ec

Luis Alejandro Montufar Calvache
Universidad de las Fuerzas Armadas ESPE
lamontufar@espe.edu.ec

Leonel Alejandro Tipán Quinto
Universidad de las Fuerzas Armadas ESPE
latipan10@espe.edu.ec

Resumen—El proyecto responde a la necesidad de observar, en una única interfaz, el comportamiento interno de un equipo GNU/Linux. La información necesaria para diagnosticar consumo de CPU, memoria, almacenamiento, red y procesos se encuentra distribuida entre archivos virtuales de `/proc`, comandos del sistema y utilidades con formatos distintos. Esta dispersión aumenta el tiempo de análisis y dificulta relacionar una anomalía general con la aplicación o el proceso que la provoca.

Index Terms—sistemas operativos, monitoreo de recursos, procesos, GNU/Linux, SQLite, CRUD, consola de comandos.

I. PROBLEMA

I-A. Contexto y necesidad

GNU/Linux expone gran parte del estado del kernel mediante interfaces como `/proc/stat`, `/proc/meminfo` y `/proc/diskstats`; además, comandos como `ps`, `who`, `df`, `free` e `ip` aportan información complementaria [1], [2]. Aunque estas fuentes son precisas, su interpretación exige conocimientos técnicos, comparación entre muestras y conversión de unidades.

El problema central puede formularse de la siguiente manera:

¿Cómo centralizar y transformar datos técnicos de GNU/Linux en información visual, histórica y persistente que facilite el análisis de recursos, aplicaciones y procesos?

I-B. Causas identificadas

- Las métricas provienen de fuentes heterogéneas y utilizan unidades diferentes.
- Varios indicadores son contadores acumulados y requieren comparar dos muestras para obtener una tasa.
- Las aplicaciones modernas pueden estar compuestas por varios procesos, por lo que analizar un solo PID produce una visión incompleta.
- Los comandos tradicionales ofrecen una fotografía puntual, pero no conservan una ventana histórica integrada.
- La evidencia de una condición desaparece cuando el sistema cambia, si no existe un mecanismo de captura persistente.
- La ejecución manual de múltiples utilidades dificulta una demostración académica ordenada y reproducible.

I-C. Consecuencias

La ausencia de una herramienta centralizada genera diagnósticos más lentos, dificultad para atribuir consumo a una aplicación específica, pérdida de contexto temporal y poca trazabilidad. Desde la perspectiva académica, también separa conceptos que deberían analizarse conjuntamente: planificación, estados de procesos, memoria virtual, entrada/salida, sistema de archivos virtual y creación de procesos.

I-D. Alcance del problema

El proyecto se concentra en monitoreo local para GNU/Linux. Incluye lectura de recursos, agrupación de aplicaciones, detalle de procesos, historial reciente, almacenamiento de capturas y evidencia de comandos. No pretende reemplazar plataformas empresariales de observabilidad, administrar varios servidores ni ejecutar acciones arbitrarias sobre el sistema.

Riesgo y control recomendado: *Una interfaz que ejecute comandos sin restricciones puede ampliar la superficie de ataque. Por ello, el diseño debe utilizar acciones predefinidas, validar parámetros y evitar concatenar texto proporcionado por el usuario en una shell.*

II. OBJETIVOS

II-A. Objetivo general

Desarrollar una aplicación web para GNU/Linux que recolecte, procese y presente métricas de recursos, aplicaciones y procesos en tiempo real, incorporando historial temporal, persistencia local y evidencia controlada de operaciones del sistema operativo.

II-B. Objetivos específicos

1. Obtener información de CPU, memoria, swap, disco y red desde fuentes nativas de GNU/Linux.
2. Calcular porcentajes y tasas mediante diferencias entre muestras consecutivas.
3. Presentar un resumen ejecutivo que permita identificar rápidamente condiciones anómalas.
4. Agrupar procesos por ejecutable para estimar el consumo total de una aplicación.

5. Mostrar el detalle individual de PID, usuario, estado, CPU, memoria y operaciones de entrada/salida.
6. Mantener series históricas de los principales indicadores dentro de una ventana temporal.
7. Implementar operaciones CRUD para conservar capturas del estado del sistema en SQLite.
8. Integrar comandos Linux predefinidos y demostraciones de `os.system()` y `fork()` como evidencia técnica.
9. Diseñar una interfaz consistente, legible y adecuada para la demostración académica.
10. Documentar riesgos, limitaciones, criterios de validación y oportunidades de evolución.

II-C. Criterios de cumplimiento

En la Tabla I se presentan los criterios de cumplimiento definidos para verificar los objetivos propuestos.

III. ARQUITECTURA

III-A. Visión general

La solución adopta una arquitectura por capas. El objetivo es separar la captura de datos del kernel, su transformación, la persistencia y la presentación. Esta división reduce acoplamiento, facilita las pruebas y permite reemplazar una fuente de métricas sin modificar toda la aplicación.

III-B. Capas y responsabilidades

En la Tabla II se detallan las responsabilidades específicas de cada una de las capas de la arquitectura del sistema.

III-C. Flujo de datos

1. El recolector obtiene una muestra del sistema operativo.
2. Los contadores se comparan con la muestra anterior.
3. La lógica convierte valores a porcentajes, bytes por segundo y unidades legibles.
4. Los procesos se conservan individualmente y también se agrupan por ejecutable.
5. El estado normalizado se publica para la interfaz y alimenta el historial.
6. Cuando el usuario crea una captura, el estado se serializa y se almacena en SQLite.

III-D. Concurrencia

El indicador visible de hilos activos sugiere que la adquisición periódica se ejecuta de forma separada del servicio de interfaz. Esta decisión evita bloquear la experiencia del usuario durante lecturas o comandos. Las estructuras compartidas deben protegerse mediante bloqueo mutuo o intercambio de instantáneas inmutables para impedir que la interfaz lea datos parcialmente actualizados.

Nota técnica: Las métricas de CPU, disco y red no se obtienen correctamente a partir de una sola lectura. Se requiere conservar la muestra anterior, calcular la diferencia y dividirla para el intervalo real transcurrido.

IV. TECNOLOGÍAS UTILIZADAS

IV-A. Resumen tecnológico

La Tabla III resume las tecnologías empleadas en el desarrollo de la aplicación y sus propósitos correspondientes.

IV-B. Fuentes nativas de GNU/Linux

- `/proc/stat`: tiempos acumulados de CPU total y por núcleo.
- `/proc/meminfo`: memoria total, disponible, caché, buffers y swap.
- `/proc/diskstats`: contadores de lectura y escritura por dispositivo.
- Directorios `/proc/[PID]`: información particular de procesos.
- `ps`: listado de procesos, usuario, estado y consumo.
- `df`: capacidad y ocupación de sistemas de archivos.
- `free`: resumen de memoria y swap.
- `ip`: direcciones, interfaces y rutas de red.
- `who`: sesiones conectadas al equipo.

IV-C. Justificación de SQLite

SQLite es adecuado para un prototipo local porque no requiere un servidor adicional, mantiene los datos en un único archivo y proporciona transacciones. La elección reduce la complejidad de despliegue y permite consultar capturas incluso después de reiniciar la aplicación. Para un escenario multiusuario o remoto sería necesario evaluar un motor cliente-servidor y una política de retención.

IV-D. Compatibilidad

La solución depende de características propias de sistemas tipo Unix. En particular, `/proc` y `fork()` no tienen equivalencia directa en Windows. La portabilidad exige abstraer los recolectores y seleccionar una implementación según el sistema operativo.

V. DESARROLLO

V-A. Metodología de construcción

El desarrollo se organizó de forma incremental. Primero se validaron las fuentes del sistema y los cálculos; posteriormente se integraron los módulos visuales, el historial, la persistencia y la evidencia de comandos. Esta secuencia reduce el riesgo de construir una interfaz atractiva sobre métricas incorrectas.

1. **Exploración:** identificación de archivos y comandos disponibles.
2. **Prototipo de recolección:** lectura de datos crudos y normalización inicial.
3. **Cálculo temporal:** almacenamiento de muestras previas y obtención de tasas.
4. **Modelo de presentación:** definición de objetos para recursos, aplicaciones y procesos.
5. **Interfaz:** construcción de navegación, tarjetas, tablas y gráficas.
6. **Persistencia:** creación del esquema SQLite y operaciones CRUD.

Cuadro I
CRITERIOS VERIFICABLES DE LOS OBJETIVOS

Código	Criterio	Evidencia esperada
C-01	El tablero muestra CPU, memoria, swap, disco y red con unidades.	Vista Overview actualizada con datos reales.
C-02	El historial conserva una secuencia temporal coherente.	Gráficas con marcas de tiempo y varias muestras.
C-03	Las aplicaciones se agrupan por ejecutable.	Tabla de aplicaciones con número de instancias.
C-04	Los procesos se visualizan individualmente.	Tabla con PID, usuario, estado y consumo.
C-05	Las capturas pueden crearse, consultarse, actualizarse y eliminarse.	Vista Records CRUD y registros SQLite.
C-06	Los comandos permitidos muestran salida y código de resultado.	Consola integrada con acciones predefinidas.
C-07	La interfaz continúa operativa si una fuente falla.	Mensaje de error localizado sin caída total.

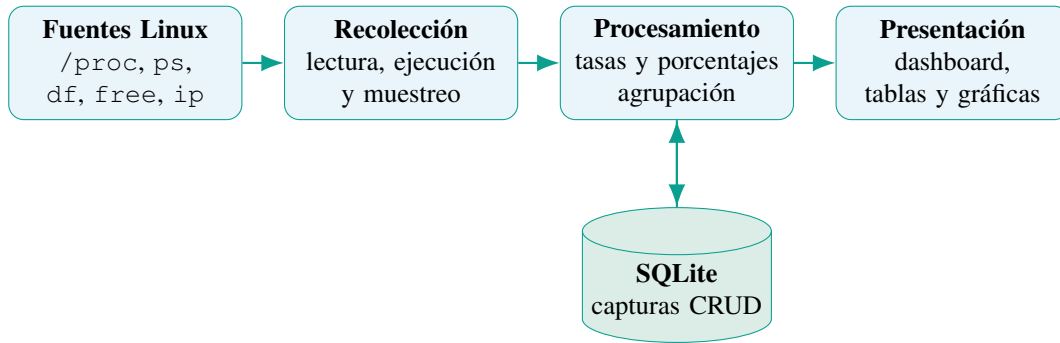


Figura 1. Arquitectura lógica del proyecto.

Cuadro II
RESPONSABILIDADES DE LA ARQUITECTURA

Capa	Responsabilidad	Entradas y salidas
Fuentes del sistema	Exponer datos del kernel y utilidades administrativas.	Archivos virtuales, procesos, dispositivos, interfaces y sesiones.
Recolección	Leer archivos y ejecutar únicamente acciones permitidas.	Texto, contadores, identificadores, tamaños y marcas de tiempo.
Procesamiento	Normalizar unidades y calcular tasas, porcentajes y agregaciones.	Objetos de CPU, memoria, disco, red, aplicaciones y procesos.
Historial	Mantener muestras ordenadas dentro de una ventana temporal.	Series para gráficas y valores de carga.
Persistencia	Conservar capturas y metadatos de manera transaccional.	Registros SQLite con etiqueta, comentario, fecha y detalle.
Presentación	Visualizar indicadores y recibir acciones del usuario.	Tarjetas, tablas, barras, gráficas, filtros y consola.

7. **Evidencia técnica:** integración de comandos predefinidos y creación de procesos.
8. **Validación:** comparación de resultados, manejo de errores y revisión visual.

multinúcleo porque cada núcleo representa hasta 100 % bajo esta convención.

V-B2. *Memoria y swap:* La memoria usada se aproxima mediante:

V-B. Tratamiento de métricas

V-B1. *CPU:* El uso total se calcula a partir de diferencias entre tiempos acumulados:

$$U_{CPU} = 100 \left(1 - \frac{\Delta T_{idle}}{\Delta T_{total}} \right) \quad (1)$$

La operación se repite para cada núcleo lógico. El uso agregado de una aplicación puede superar 100 % en equipos

$$M_{usada} = M_{total} - M_{disponible}, \%M = 100 \frac{M_{usada}}{M_{total}} \quad (2)$$

Se muestran caché y buffers por separado para evitar interpretar toda memoria ocupada como presión real. La swap se calcula de forma independiente.

V-B3. *Disco y red:* Las tasas se obtienen con el mismo patrón de diferencia temporal:

Cuadro III
TECNOLOGÍAS Y PROPÓSITO DENTRO DEL PROYECTO

Tecnología	Categoría	Uso principal
GNU/Linux	Plataforma	Fuente de métricas, procesos, usuarios, red y sistema de archivos.
Python	Backend y recolección	Lectura de archivos, cálculos, concurrencia, persistencia y ejecución de procesos.
/proc subprocess	Interfaz del kernel Biblioteca estándar	CPU, memoria, procesos y contadores de actividad. Ejecución controlada de comandos y captura de salida [3].
os	Biblioteca estándar	Interfaces del sistema, demostración de os.system() y fork() [4].
SQLite	Base de datos embebida	Almacenamiento local de capturas y operaciones CRUD [5].
HTML, CSS y JavaScript	Interfaz web	Estructura, apariencia, interacción y actualización visual.
Gráficas en navegador	Visualización	Serie temporales de CPU, memoria, red, disco, swap y carga.
Git y GitHub	Control de versiones	Historial del código y distribución del repositorio.

$$R = \frac{C(t_2) - C(t_1)}{t_2 - t_1} \quad (3)$$

En disco, C representa sectores o bytes transferidos; en red, bytes recibidos o enviados. El intervalo debe medirse y no suponerse constante, porque una actualización tardía afectaría la exactitud.

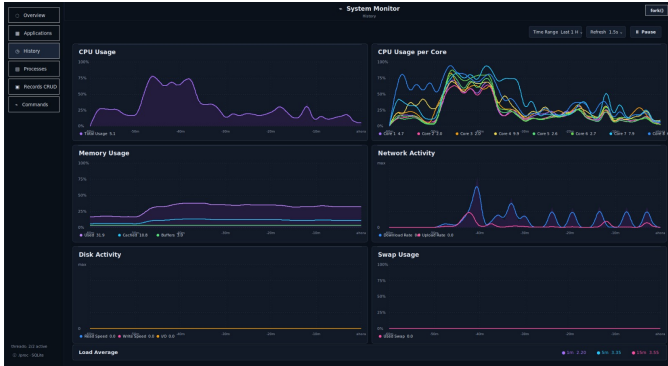


Figura 2. Historial temporal de recursos.

Las gráficas muestran CPU total, uso por núcleo, memoria, red, disco, swap y carga. La presencia de varias muestras demuestra que el sistema mantiene una ventana histórica y permite relacionar picos de consumo con el momento en que ocurrieron.

V-C. Módulos implementados

La Tabla IV describe de forma funcional los módulos integrados en la interfaz del proyecto.

V-D. Agrupación de aplicaciones

Los procesos se agrupan por nombre de ejecutable. Para cada grupo se acumulan instancias, CPU, memoria residente y E/S. Esta técnica permite observar el costo real de programas multiproceso, como navegadores o entornos gráficos, sin perder el detalle individual disponible en la tabla de procesos.

Listing 1. Pseudocódigo de agrupación no ejecutable

```
applications = {}
for process in processes:
    name = process["name"]
    app = applications.setdefault(name, {
        "instances": 0,
        "cpu": 0.0,
        "rss": 0,
        "read": 0,
        "write": 0,
    })
    app["instances"] += 1
    app["cpu"] += process["cpu"]
    app["rss"] += process["rss"]
    app["read"] += process["read"]
    app["write"] += process["write"]
```

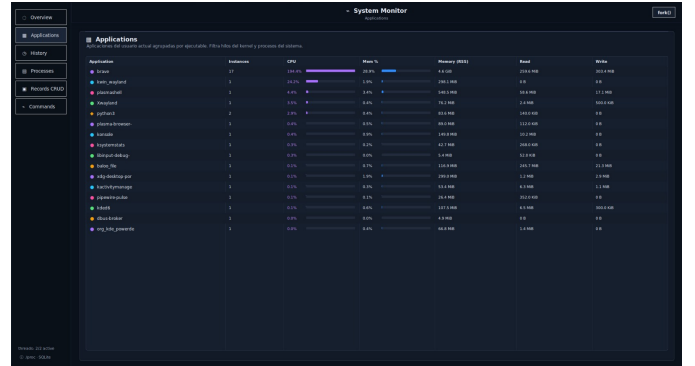


Figura 3. Módulo de aplicaciones agrupadas.

La tabla agrupa procesos por ejecutable e informa instancias, CPU acumulada, porcentaje de memoria, RSS y operaciones de lectura/escritura. La evidencia confirma que aplicaciones multiproceso pueden analizarse como una unidad sin eliminar el detalle por PID.

V-E. Persistencia y CRUD

Cada captura conserva metadatos legibles y un detalle estructurado. En la Tabla V se presenta el diseño del modelo lógico mínimo de persistencia.

Cuadro IV
DESCRIPCIÓN FUNCIONAL DE LOS MÓDULOS

Módulo	Función	Datos relevantes
Overview	Resumen ejecutivo del equipo.	CPU, memoria, swap, disco, red, carga, tiempo activo y aplicaciones principales.
Applications	Agrupación por ejecutable.	Instancias, CPU total, porcentaje de memoria, RSS y E/S.
History	Análisis temporal.	CPU total y por núcleo, memoria, red, disco, swap y carga.
Processes	Inspección por PID.	Usuario, estado, nombre, CPU, memoria, lectura y escritura.
Records CRUD	Persistencia de instantáneas.	Etiqueta, comentario, fecha, resumen y detalle estructurado.
Commands	Evidencia del sistema.	ps, who, df, free, ip, os.system() y fork().

Cuadro V
MODELO LÓGICO DE UNA CAPTURA

Campo	Tipo lógico	Descripción
id	Entero	Identificador único autoincremental.
created_at	Fecha/hora	Momento exacto de creación.
label	Texto	Nombre corto definido por el usuario.
comment	Texto	Observación asociada a la captura.
payload	JSON/texto	Estado completo de recursos y procesos.

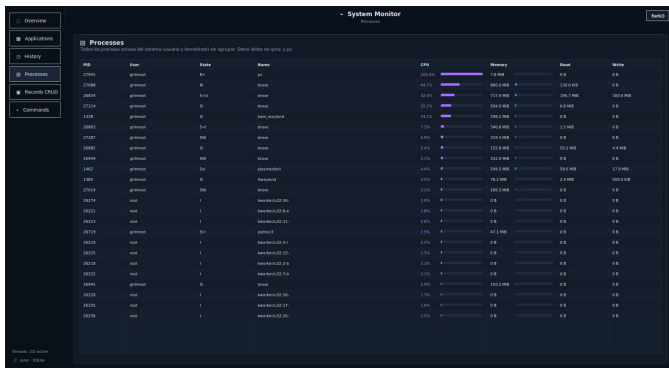


Figura 4. Módulo de procesos del sistema.

La vista lista PID, usuario, estado, nombre, CPU, memoria y E/S. Se observan procesos del usuario y del kernel, lo que permite estudiar planificación, estados y distribución de recursos con un nivel de detalle individual.

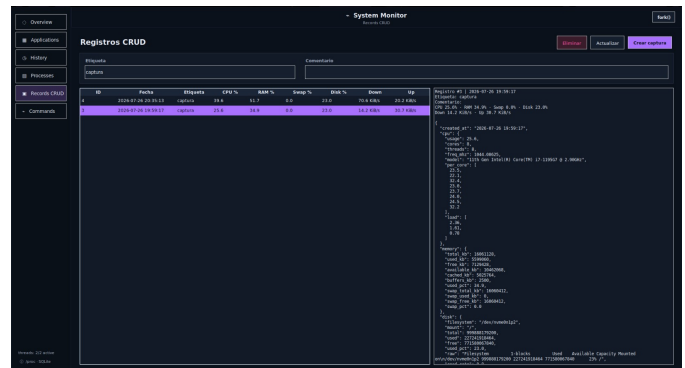


Figura 5. Registros persistentes y operaciones CRUD.

La pantalla presenta capturas almacenadas con ID, fecha, etiqueta y resumen de métricas. El panel derecho despliega el detalle estructurado del registro seleccionado. Los botones de crear, actualizar y eliminar evidencian el ciclo CRUD respaldado por SQLite.

V-F. Ejecución de comandos

La ejecución se restringe a una lista de acciones conocidas. subprocess permite capturar salida estándar, salida de error, tiempo de espera y código de retorno. os.system() se mantiene únicamente como demostración académica controlada; para funcionalidad general resulta preferible evitar una shell intermedia.

```
command = ALLOWED_COMMANDS[action]
result = subprocess.run(
    command,
    capture_output=True,
    text=True,
    timeout=5,
    check=False,
)
```

Listing 2. Patrón seguro para comandos predefinidos

```
ALLOWED_COMMANDS = {
    "ps": ["ps", "aux"],
    "who": ["who", "-u"],
    "df": ["df", "-h"],
    "free": ["free", "-h"],
    "ip": ["ip", "address"],
}
```

V-G. Manejo de errores y seguridad

- Validar que las fuentes existan y sean legibles antes de procesarlas.
- Capturar errores por módulo para que una falla no detenga todo el tablero.
- Definir tiempo máximo para comandos externos.

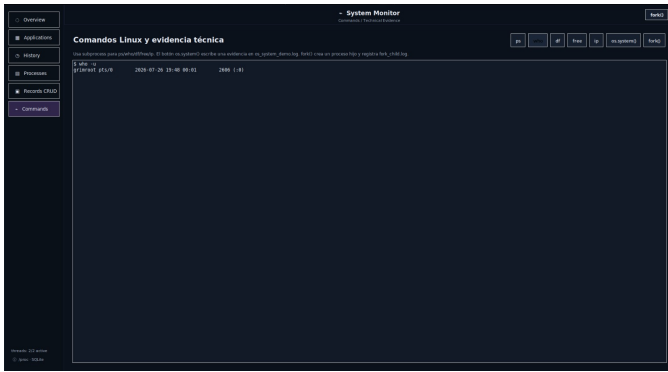


Figura 6. Comandos Linux y evidencia técnica.

La consola integra acciones para `ps`, `who`, `df`, `free` e `ip`, además de demostraciones de `os.system()` y `fork()`. La captura confirma la ejecución controlada y la presentación de resultados dentro de la interfaz.

- No aceptar comandos arbitrarios ni parámetros concatenados.
- Limitar el tamaño del historial y de los payloads almacenados.
- Registrar fecha, módulo y detalle de fallos internos.
- Ejecutar con los permisos mínimos necesarios.

Nota técnica: En esta sección se documenta el proceso de construcción mediante texto, tablas, fórmulas y fragmentos técnicos. Las capturas de la aplicación se presentan en sus respectivas secciones para mantener la coherencia y una adecuada visualización de la evidencia técnica.

VI. RESULTADOS

VI-A. Resultado funcional

El prototipo integra en una sola aplicación seis módulos complementarios. La navegación permite pasar de un resumen general a un análisis por aplicación o proceso, observar la evolución temporal, conservar capturas y ejecutar demostraciones técnicas. La solución cumple el propósito académico de relacionar conceptos internos del sistema operativo con datos visibles y verificables.

VI-B. Métricas observadas durante la ejecución

La Tabla VI presenta las métricas obtenidas durante una prueba real de funcionamiento del proyecto.

VI-C. Cumplimiento de objetivos

La Tabla VII detalla el estado y la justificación del nivel de cumplimiento alcanzado para cada meta del proyecto.

VI-D. Fortalezas

- Interfaz coherente y orientada a lectura rápida.
- Integración de múltiples conceptos de Sistemas Operativos.
- Distinción entre vista agregada y detalle por proceso.
- Persistencia de evidencia mediante capturas CRUD.
- Uso de datos reales del equipo y no de valores simulados.
- Base extensible para alertas, exportación y monitoreo remoto.



Figura 7. Vista general del proyecto.

La pantalla consolida CPU, memoria, swap, red y aplicaciones principales. También muestra núcleos, hilos, procesos, carga promedio, interfaz de red, direcciones IP y tiempo activo. Esta vista valida la integración de varias fuentes de GNU/Linux en un resumen único.

VI-E. Limitaciones y mejoras

En la Tabla VIII se enumeran las principales limitaciones del sistema y sus respectivas propuestas de desarrollo futuro.

Hallazgo: La principal contribución del proyecto es convertir mecanismos internos del sistema operativo en una secuencia de observación comprensible: resumen, historial, aplicación, proceso y evidencia persistente.

VII. CONCLUSIONES

1. El proyecto centraliza información que normalmente se consulta mediante múltiples archivos y comandos, reduciendo la fragmentación del análisis.
2. La arquitectura por capas separa recolección, procesamiento, persistencia y presentación, lo que favorece mantenimiento y pruebas.
3. El uso de diferencias entre muestras permite transformar contadores acumulados de CPU, disco y red en porcentajes y tasas comprensibles.
4. La combinación de Applications y Processes ofrece una visión complementaria: consumo agregado por programa y detalle individual por PID.
5. El historial temporal mejora la interpretación porque conserva contexto y permite identificar picos que una lectura puntual no mostraría.
6. La persistencia SQLite añade trazabilidad al permitir conservar, consultar, modificar y eliminar capturas del estado del equipo.
7. La integración de subprocess, os.system() y fork() vincula la interfaz con conceptos de creación y ejecución de procesos, siempre que se mantengan controles de seguridad.
8. Las principales oportunidades de evolución son alertas, exportación, autenticación, retención histórica, despliegue como servicio y monitoreo remoto.

Conclusión principal: El proyecto cumple una función doble: es una herramienta de monitoreo local y, al mismo tiempo, una representación práctica de conceptos de Sistemas Operativos.

Cuadro VI
VALORES VISIBLES EN LA MUESTRA PRINCIPAL

Indicador	Valor observado	Interpretación
CPU total	15 % a 1.32 GHz	Carga moderada durante la captura.
Núcleos / hilos	8 / 8	Ocho unidades lógicas disponibles.
Procesos	80	Actividad combinada de usuario y kernel.
Carga promedio	2.94	Trabajo ejecutable o pendiente dentro de la ventana de carga.
Memoria	5.2 GiB de 15.3 GiB	Uso aproximado de 34 % con capacidad disponible.
Swap	0 %	Sin presión visible de intercambio.
Disco raíz	211.7 GiB de 931.2 GiB	Ocupación aproximada de 23 %.
Red	5.5 KiB/s abajo; 6.5 KiB/s arriba	Tráfico bajo en el instante de observación.
Tiempo activo	0 h 53 min 45 s	Tiempo transcurrido desde el arranque.

Cuadro VII
EVALUACIÓN DEL CUMPLIMIENTO

Resultado	Estado	Justificación
Monitoreo de recursos	Cumplido	Se visualizan CPU, memoria, swap, disco y red.
Aplicaciones agrupadas	Cumplido	Se suman instancias, CPU, RSS y E/S por ejecutable.
Detalle de procesos	Cumplido	Se presentan PID, usuario, estado y consumo.
Historial temporal	Cumplido	Existen series para recursos y núcleos.
Persistencia CRUD	Cumplido	Las capturas pueden administrarse en SQLite.
Comandos y creación de procesos	Cumplido	Se integran comandos predefinidos y evidencia de <code>fork()</code> .
Monitoreo remoto	Fuera de alcance	La versión analizada opera sobre el host local.
Alertas automáticas	Pendiente	Se propone como evolución posterior.

Cuadro VIII
LIMITACIONES Y EVOLUCIÓN RECOMENDADA

Limitación	Impacto	Mejora propuesta
Dependencia de GNU/Linux	No existe portabilidad directa a Windows.	Implementar recolectores por plataforma.
Historial en memoria	Se pierde al cerrar la aplicación.	Persistir series o exportarlas periódicamente.
Monitoreo local	Solo se observa el host de ejecución.	Crear un agente remoto y panel multi-host.
Ausencia de alertas	El usuario debe revisar manualmente.	Definir umbrales y notificaciones.
Payloads extensos	La base puede crecer rápidamente.	Aplicar retención, compresión y separación de detalle.
Comandos del sistema	Pueden variar entre distribuciones.	Detectar disponibilidad y normalizar salidas.

Su mayor valor consiste en hacer observable la relación entre kernel, procesos, memoria, entrada/salida, aplicaciones y persistencia.

REFERENCIAS

- [1] The Linux Kernel Documentation, “The /proc Filesystem.” [En línea]. Disponible en: <https://docs.kernel.org/filesystems/proc.html>.
- [2] M. Kerrisk, “proc(5) – process information pseudo-filesystem.” [En línea]. Disponible en: <https://man7.org/linux/man-pages/man5/proc.5.html>.
- [3] Python Software Foundation, “subprocess – Subprocess management.” [En línea]. Disponible en: <https://docs.python.org/3/library/subprocess.html>.
- [4] Python Software Foundation, “os – Miscellaneous operating system interfaces.” [En línea]. Disponible en: <https://docs.python.org/3/library/os.html>.
- [5] SQLite Consortium, “About SQLite.” [En línea]. Disponible en: <https://www.sqlite.org/about.html>.
- [6] A. Silberschatz, P. B. Galvin, y G. Gagne, *Operating System Concepts*, 10.^a ed. Wiley, 2018.